

1. explicación y análisis

tengo un función booleana recursiva la cual me identifica que si un valor del array es mayor al siguiente

la función compara un elemento con el siguiente $\text{if} (\text{array}[i] > \text{array}[i + 1])$

y si el elemento actual es mayor al siguiente elemento retorna false , por que significa que no esta ordenado de mayor a menor , si no pasa eso entonces se vuelve a llamar a la función agregando uno a mi indice que vendría a ser i , este proceso se repite hasta llegar al ultimo Si durante las comparaciones no se encuentra ningún elemento mayor que el siguiente, la función retorna true, indicando que el arreglo sí está ordenado correctamente.

2. Explicar el caso base.

mi caso base es

```
if (i == n ){  
    return true ;  
}
```

esto hace que , si mi indice i es igual al tamaño de mi array osea el ultimo elemento y si no se encontro algun elemento que sea mayor al siguiente , la función retorna true .

3. obtener la recurrencia $T(n)$

$$T(n) = T(n-1) + 1$$

se realizan comparaciones constantes y en cada llamada se reduce un elemento

4. la complejidad temporal.

cada llamada hace una comparación y una llamada recursiva:

y eso pasa n veces hasta que $i = n$ o hasta que $i \geq i+1$ por eso es :

$$O(n)$$